

一脑多控平台详细设计说明书 V1.0

1. 项目定位

本平台面向多型号机器狗、轮式机器人、仿真机器人等端侧设备，构建一套“云端大脑 + 边缘安全执行 + 多机器人协同”的一脑多控系统。

平台目标不是让大模型直接控制机器人底层接口，而是将自然语言任务转换为可验证、可审计、可回放、可安全执行的机器人任务链。

推荐总体定位为：

一脑多控平台 =
云端任务规划与多机调度平台
+ Agent Runtime
+ MCP/Skill 工具体系
+ ROSClaw Edge Runtime
+ ROS1/ROS2/Unitree/私有协议适配层
+ Safety Gateway
+ Trace/Memory 闭环

ROSClaw 官方定位是面向 Physical AI 和 embodied agents 的 runtime infrastructure，强调 e-URDF、sandbox safety、capability routing、physical memory、praxis capture、runtime intervention 和 skill evolution，因此更适合作为边缘侧机器人运行时底座参考。AgenticROS 官方定位更偏 “Agentic ROS plugin”，用于把 OpenClaw、Claude、Codex、Gemini、MCP 等 AI Agent 接入 ROS2 机器人，适合作为 ROS2 快速接入参考，而不是完整一脑多控平台底座。

2. 总体架构

用户 / 管理后台 / 指挥端

↓

一脑多控云中心

- Agent Runtime: OpenClaw / NemoClaw / Hermes / QwenPaw
- Mission Planner: 任务规划
- Fleet Orchestrator: 多机器人调度
- Skill Registry: 技能注册
- Robot Registry: 机器人注册
- Policy Center: 权限与安全策略
- Memory Center: 任务记忆 / 失败经验
- Trace Center: 执行轨迹 / 审计日志
- MCP Tool Registry: 工具注册中心
- Map & Scene Center: 地图 / 场景 / 区域管理
- Media Center: 视频流 / 图片 / 传感器数据接入
- Ops Center: 监控 / 告警 / OTA / 运维

↓ MCP / HTTP / WebSocket / Zenoh / gRPC

边缘机器人网关

- ├── ROSClaw Edge Runtime
- ├── Safety Gateway
- ├── ROS2 Adapter
- ├── ROS1 Adapter
- ├── Unitree SDK Adapter
- ├── Custom Protocol Adapter
- ├── Camera / LiDAR / IMU Adapter
- ├── Local E-Stop
- ├── Local Skill Executor
- ├── Local Telemetry Collector
- └── Local Cache / Offline Queue

↓

端侧机器人

- ├── ROS2 机器狗
- ├── ROS1 机器狗
- ├── Unitree SDK 机器狗
- ├── 自研协议机器狗
- └── 仿真机器人

核心原则：

云端负责：理解任务、拆解任务、选择机器人、全局调度、记录知识。

边缘负责：能力适配、安全检查、实时执行、异常保护、断网兜底。

机器人负责：底层运动控制、传感器采集、本体状态反馈。

3. 设计原则

3.1 大模型不直接控制底层接口

禁止让大模型直接发布：

```
/cmd_vel  
/joint_trajectory  
/low_level_control  
/motor_command  
/udp_raw_command
```

大模型只能调用经过注册的高层能力：

```
stand_up()  
lie_down()
```

```
stop()
move_forward(distance, speed)
turn(angle, speed)
navigate_to(target)
inspect_area(area_id)
capture_image(camera_id)
return_home()
emergency_stop()
```

3.2 所有动作必须经过安全网关

所有动作执行前必须经过：

权限校验
任务策略校验
设备状态校验
环境风险校验
速度 / 距离 / 时间限制
电量 / 姿态 / 传感器状态检查
急停优先级判断

3.3 云边解耦

云端可以断连，边缘网关仍必须具备：

本地急停
本地停止
本地状态监控
任务中断
安全降级
断点续传
日志缓存

3.4 能力抽象优先于设备型号

不同机器狗可能来自不同厂商、不同 ROS 版本、不同 SDK。平台不直接面向设备型号编排任务，而是面向能力编排任务：

能力：navigate、inspect、capture、follow、patrol、stand、stop
设备：Unitree Go2、ROS1 Dog、ROS2 Dog、自研机器狗

4. 技术选型建议

层级	推荐技术	说明
云端后端	Java / Go / Python 混合	管理类服务用 Java/Go，Agent/机器人适配用 Python
前端	Vue / React + WebSocket	指挥端、地图、设备状态、任务监控
Agent Runtime	OpenClaw / NemoClaw / Hermes / QwenPaw	作为大脑执行环境
工具协议	MCP 优先，HTTP/gRPC 辅助	MCP 适合 Agent 调用工具
边缘运行时	ROSClaw Edge Runtime	负责物理能力、安全沙盒、执行轨迹
ROS2 接入	ROS2 DDS / rosbridge / Zenoh	ROS2 主路径
ROS1 接入	ros1_bridge / rosbridge_suite / 自研 ROS1 Gateway	ROS1 作为兼容路径
弱网通信	Zenoh	Zenoh 是 pub/sub/query 协议，适合分布式边缘通信；其 ROS2DDS bridge 可桥接 ROS2 DDS 通信。
数据库	PostgreSQL + TimescaleDB 可选	设备、任务、策略、轨迹、遥测
对象存储	MinIO / S3	图片、视频、地图、日志包、bag 文件
向量库	pgvector / Milvus	任务记忆、失败案例、场景语义检索
消息队列	Kafka / NATS / RabbitMQ	任务事件、状态事件、告警事件
监控	Prometheus + Grafana	服务、设备、边缘网关监控
日志	Loki / Elasticsearch	指令审计、任务日志、异常日志

5. 云中心详细设计

5.1 用户与指挥端

功能

设备地图展示
机器人实时状态
任务创建
任务模板管理
任务执行进度
视频/图片查看
手动接管
急停按钮
日志回放
告警查看

页面模块

1. 首页大屏
2. 设备管理
3. 任务管理
4. 技能管理
5. 地图与区域管理
6. 实时监控
7. 告警中心
8. 审计日志
9. 系统配置

指挥端关键能力

自然语言下发任务：

“让 1 号机器狗去 A 区巡检，发现异常拍照并返回。”

结构化任务下发：

选择机器人 → 选择区域 → 选择技能 → 设置安全策略 → 执行。

人工接管：

暂停任务、恢复任务、停止任务、急停、返航。

5.2 Agent Runtime

定位

Agent Runtime 负责把用户自然语言、任务模板、环境状态转换成可执行任务计划。

可选运行时：

OpenClaw
NemoClaw
Hermes
QwenPaw

在平台中不要把某一个 Agent 框架写死，应抽象为：

```
AgentRuntimeProvider
├── OpenClawProvider
├── NemoClawProvider
├── HermesProvider
└── QwenPawProvider
```

输入

```
{
  "user_command": "让机器狗去A区巡检，发现异常拍照并返回",
  "available_robots": ["dog-001", "dog-002"],
  "available_skills": ["patrol_area", "capture_image", "return_home"],
  "map_context": "factory-floor-1",
  "policy_context": "normal_inspection_policy"
}
```

输出

```
{
  "mission_name": "A区巡检任务",
  "steps": [
    {
      "skill": "navigate_to",
      "target_robot": "dog-001",
      "params": {
        "area_id": "area-a"
      }
    },
    {
      "skill": "inspect_area",
      "target_robot": "dog-001",
      "params": {
        "area_id": "area-a",
        "capture_on_anomaly": true
      }
    },
    {
      "skill": "return_home",
      "target_robot": "dog-001",
      "params": {}
    }
  ]
}
```

5.3 Mission Planner：任务规划中心

职责

```
自然语言任务解析
任务模板匹配
任务 DAG 生成
任务参数补全
```

任务风险评估
任务可行性校验
任务回滚计划生成

任务 DAG 示例

```
mission_id: mission-20260703-001
name: A区巡检
priority: normal
timeout: 1800
robots:
  - dog-001
steps:
  - id: step-1
    type: skill
    skill: stand_up
    robot: dog-001

  - id: step-2
    type: skill
    skill: navigate_to
    robot: dog-001
    depends_on: [step-1]
    params:
      target: area-a-entry

  - id: step-3
    type: skill
    skill: inspect_area
    robot: dog-001
    depends_on: [step-2]
    params:
      area_id: area-a
      anomaly_capture: true

  - id: step-4
    type: skill
    skill: return_home
    robot: dog-001
    depends_on: [step-3]
```

状态机

```
CREATED
→ PLANNING
→ WAITING_APPROVAL
→ SCHEDULED
→ DISPATCHED
→ RUNNING
```

- PAUSED
- COMPLETED
- FAILED
- CANCELLED
- EMERGENCY_STOPPED

5.4 Fleet Orchestrator：多机器人调度中心

职责

- 机器人选择
- 多机器人任务分配
- 机器人负载均衡
- 路径冲突检测
- 任务锁管理
- 资源占用管理
- 故障转移
- 任务重分配

调度因子

- 机器人在线状态
- 电量
- 当前位置
- 设备类型
- 支持能力
- 传感器配置
- 当前任务负载
- 区域权限
- 历史成功率
- 故障状态
- 网络质量

调度策略

- 最近距离优先
- 电量最高优先
- 能力匹配优先
- 空闲机器人优先
- 指定型号优先
- 任务成功率优先
- 人工指定优先

机器人选择示例

```
{
  "task": "inspect_area",
  "required_capabilities": ["navigate", "camera", "obstacle_avoidance"],
  "candidate_robots": [
    {
      "robot_id": "dog-001",
      "score": 92,
      "reason": "距离近、电量高、支持巡检和拍照"
    },
    {
      "robot_id": "dog-002",
      "score": 76,
      "reason": "距离较远，但能力满足"
    }
  ],
  "selected_robot": "dog-001"
}
```

5.5 Robot Registry：机器人注册中心

职责

机器人注册
型号管理
网关绑定
能力声明
状态维护
分组管理
地图绑定
传感器配置
维护状态
证书管理

核心数据模型

Robot

- robot_id
- robot_name
- robot_type
- vendor
- model
- ros_version
- control_protocol
- edge_gateway_id

- └── online_status
- └── battery_level
- └── location
- └── capabilities
- └── sensors
- └── safety_profile
- └── created_at

注册请求示例

```
{
  "robot_id": "dog-001",
  "robot_name": "A区巡检机器狗1号",
  "vendor": "Unitree",
  "model": "Go2",
  "robot_type": "quadruped",
  "ros_version": "ros2",
  "control_protocol": "unitree_sdk",
  "edge_gateway_id": "edge-001",
  "capabilities": [
    "stand",
    "walk",
    "turn",
    "navigate",
    "capture_image",
    "inspect_area",
    "return_home"
  ],
  "sensors": [
    "camera",
    "imu",
    "lidar"
  ]
}
```

5.6 Skill Registry: 技能注册中心

技能分类

基础运动技能

- └── stand_up
- └── lie_down
- └── stop
- └── move_forward
- └── turn
- └── return_home

导航技能

- └─ navigate_to
- └─ patrol_route
- └─ inspect_area
- └─ avoid_obstacle

感知技能

- └─ capture_image
- └─ detect_object
- └─ read_gauge
- └─ inspect_light
- └─ detect_abnormal

协同技能

- └─ follow_robot
- └─ multi_robot_patrol
- └─ relay_inspection
- └─ surround_observe

安全技能

- └─ emergency_stop
- └─ safe_shutdown
- └─ low_battery_return
- └─ fall_recovery

技能定义

```
skill_id: inspect_area
name: 区域巡检
version: 1.0.0
category: inspection
description: 控制机器人在指定区域内巡检并采集异常图像
required_capabilities:
- navigate
- camera
- obstacle_avoidance
input_schema:
  area_id:
    type: string
    required: true
  anomaly_capture:
    type: boolean
    default: true
safety_constraints:
  max_speed: 0.6
  max_duration: 1800
  require_obstacle_check: true
  require_battery_above: 30
supported_robot_types:
- quadruped
```

```
- wheeled
executor:
  type: edge
  entry: local_skill_executor.inspect_area
```

5.7 MCP Tool Registry: 工具注册中心

职责

管理 Agent 可调用工具
将机器人技能暴露为 MCP Tool
控制工具权限
维护工具 Schema
记录工具调用日志
支持工具灰度发布

MCP 工具示例

```
{
  "tool_name": "robot.inspect_area",
  "description": "控制指定机器人巡检指定区域",
  "input_schema": {
    "type": "object",
    "properties": {
      "robot_id": {
        "type": "string"
      },
      "area_id": {
        "type": "string"
      },
      "capture_on_anomaly": {
        "type": "boolean"
      }
    },
    "required": ["robot_id", "area_id"]
  },
  "policy": {
    "requires_approval": true,
    "roles_allowed": ["admin", "dispatcher"],
    "safety_level": "medium"
  }
}
```

工具调用链路

```
Agent
  ↓ MCP Tool Call
MCP Tool Registry
  ↓ 权限校验
Mission Planner
  ↓ 任务生成
Fleet Orchestrator
  ↓ 调度
Edge Gateway
  ↓ 安全执行
Robot
```

5.8 Policy Center: 权限与安全策略中心

策略类型

```
用户权限策略
机器人操作策略
任务审批策略
区域访问策略
速度限制策略
时间窗口策略
危险动作策略
人工确认策略
急停策略
```

策略示例

```
policy_id: normal_inspection_policy
name: 普通巡检策略
scope: inspection
rules:
  - name: 电量限制
    condition: robot.battery < 30
    action: block

  - name: 最大速度限制
    condition: action.speed > 0.6
    action: modify
    value:
      speed: 0.6

  - name: 非工作时间需要审批
    condition: current_time not_in work_hours
```

```
action: require_approval
```

```
- name: 禁止进入危险区域  
  condition: target_area in restricted_areas  
  action: block
```

5.9 Memory Center：任务记忆与失败经验中心

记忆类型

任务记忆：某类任务如何执行
环境记忆：某区域地形、障碍物、风险点
机器人记忆：某型号机器狗的特性
失败记忆：失败原因、恢复动作、规避策略
技能记忆：技能执行效果、成功率、适配设备

记忆写入来源

任务执行轨迹
机器人状态
Agent 推理过程
用户反馈
异常事件
安全拦截事件
人工干预记录

失败经验示例

```
{  
  "memory_type": "failure_case",  
  "robot_model": "Unitree Go2",  
  "skill": "turn",  
  "scene": "湿滑地面",  
  "failure": "转弯时出现打滑",  
  "cause": "速度过高，地面摩擦不足",  
  "fix": "将转弯速度限制为0.2m/s，并开启姿态稳定检查",  
  "success_rate_after_fix": 0.93  
}
```

5.10 Trace Center：执行轨迹与审计中心

记录内容

用户指令
Agent 推理结果
任务 DAG
机器人选择过程
每一步技能调用
安全校验结果
边缘执行日志
机器人状态变化
图片/视频/传感器数据
异常与告警
人工干预记录
最终结果

审计链路

谁下发了任务
Agent 如何理解任务
平台选择了哪台机器人
为什么选择这台机器人
执行了哪些技能
每个技能参数是什么
安全网关是否修改/拦截
机器人实际执行结果如何
任务是否成功
失败原因是什么

6. 边缘机器人网关详细设计

6.1 边缘网关职责

边缘网关是平台的关键安全边界，不能只是一个透传服务。

核心职责

连接云中心
连接本地机器人
执行本地技能
本地安全校验
本地急停
状态采集
日志缓存
断网降级

协议转换
传感器数据接入

边缘网关模块

Edge Gateway

- ├── Cloud Connector
- ├── ROSClaw Edge Runtime
- ├── Safety Gateway
- ├── Skill Executor
- ├── Adapter Manager
- ├── Telemetry Collector
- ├── Media Collector
- ├── Local Policy Engine
- ├── Local E-Stop Controller
- ├── Local Cache
- └── Watchdog

6.2 ROSClaw Edge Runtime

定位

ROSClaw Edge Runtime 用于把云端任务转换为本地可执行能力，并记录物理世界执行轨迹。ROSClaw 官方强调它不是薄 LLM-to-ROS wrapper，而是将 AI agent、robot embodiment、simulation sandbox、capability routing、physical memory、praxis capture、runtime intervention、skill evolution 连接成统一运行层。

运行流程

```
graph TD; A[收到云端任务] --> B[解析技能调用]; B --> C[获取机器人身体上下文]; C --> D[查询能力 Provider]; D --> E[进入 Safety Gateway]; E --> F[执行本地技能]; F --> G[采集状态与轨迹]; G --> H[生成执行结果]; H --> I[上传 Trace / Memory];
```


6.3 Safety Gateway

安全检查项

1. 当前机器人是否在线
2. 是否处于可执行状态
3. 是否已站立
4. 电量是否满足要求
5. 速度是否超过限制
6. 距离是否超过限制
7. 目标区域是否允许进入
8. 是否存在障碍物
9. 是否存在跌倒风险
10. 是否触发急停
11. 是否需要人工确认
12. 是否在允许时间窗口

安全动作

ALLOW: 允许执行
BLOCK: 阻止执行
MODIFY: 修改参数后执行
REQUIRE_CONFIRMATION: 要求人工确认
EMERGENCY_STOP: 立即急停

示例

```
{
  "action": "move_forward",
  "input": {
    "distance": 5.0,
    "speed": 1.2
  },
  "safety_result": {
    "decision": "MODIFY",
    "reason": "速度超过普通巡检策略限制",
    "modified_input": {
      "distance": 5.0,
      "speed": 0.6
    }
  }
}
```

6.4 ROS2 Adapter

接入方式

```
ROS2 DDS  
rosbridge_server  
Zenoh ROS2DDS bridge  
AgenticROS 参考插件
```

Zenoh 的 ROS2DDS bridge 可桥接 ROS2 DDS 通信，并提供更好的 ROS graph 集成、ROS topics/services/actions 跨 bridge 可见、ROS namespace 配置等能力。

能力映射

```
move_forward()  
→ publish /cmd_vel  
  
navigate_to()  
→ call Nav2 action  
  
capture_image()  
→ subscribe /camera/image_raw  
  
get_battery()  
→ subscribe /battery_state  
  
get_pose()  
→ subscribe /odom 或 /tf
```

6.5 ROS1 Adapter

ROS1 不建议作为新项目主路线，但现实中旧设备可能大量存在。`ros1_bridge` 官方说明它用于 ROS1 与 ROS2 之间交换消息，但自定义 message/service 类型通常需要从源码构建并同时 source ROS1/ROS2 工作空间；此外 ROS Noetic 已在 2025 年 5 月 EOL，Ubuntu 24.04 也不支持 ROS1，因此 ROS1 应作为兼容层而不是长期主栈。

推荐接入方案

方案一：ROS1 → ROS2 桥接

```
ROS1 Robot  
↓  
ros1_bridge  
↓  
ROS2 Interface
```

↓
ROSClaw Edge Runtime

方案二：ROS1 直接网关

ROS1 Robot
↓
rosbridge_suite / ROS1 Gateway
↓
Edge Gateway
↓
云中心

适配建议

标准消息优先桥接
自定义消息统一映射为平台 Capability
避免平台直接依赖 ROS1 topic
旧设备逐步封装成 Hardware MCP

6.6 Unitree SDK Adapter

职责

封装 Unitree SDK / UDP 指令
隐藏底层控制码
提供统一高层能力
做姿态与安全校验
回传本体状态

能力映射

```
stand_up()  
→ Unitree SDK stand command  
  
lie_down()  
→ Unitree SDK damp / down command  
  
move_forward(distance, speed)  
→ SDK velocity command + duration control  
  
turn(angle, speed)  
→ SDK yaw command  
  
stop()
```

→ SDK zero velocity / damping

get_status()

→ SDK state feedback

注意

Unitree 机器狗不能把底层 UDP 指令直接暴露给大模型。所有 SDK 调用都必须通过 Safety Gateway 和 Local Skill Executor。

6.7 Local Skill Executor

职责

执行基础技能
执行组合技能
管理技能生命周期
处理超时
处理失败重试
上传技能结果

技能执行状态

PENDING
VALIDATING
RUNNING
PAUSED
SUCCEEDED
FAILED
TIMEOUT
CANCELLED
EMERGENCY_STOPPED

本地技能示例

```
def move_forward(distance: float, speed: float):  
    safety.check_robot_ready()  
    safety.check_speed(speed)  
    safety.check_distance(distance)  
    adapter.send_velocity(speed, 0, 0)  
    wait_until_distance_reached(distance)  
    adapter.stop()  
    trace.record("move_forward_completed")
```

7. 多型号机器狗统一能力模型

7.1 能力抽象



7.2 设备适配矩阵

设备类型	控制方式	推荐适配
ROS2 机器狗	ROS2 DDS / Nav2 / rosbridge	ROS2 Adapter
ROS1 机器狗	ROS1 topic/service	ROS1 Gateway / ros1_bridge
Unitree 机器狗	Unitree SDK / UDP	Unitree SDK Adapter
自研协议机器狗	TCP/UDP/串口/HTTP	Custom Protocol Adapter
仿真机器人	Gazebo / Isaac Sim / TurtleBot3	Simulation Adapter

8. 通信设计

8.1 云中心到边缘网关

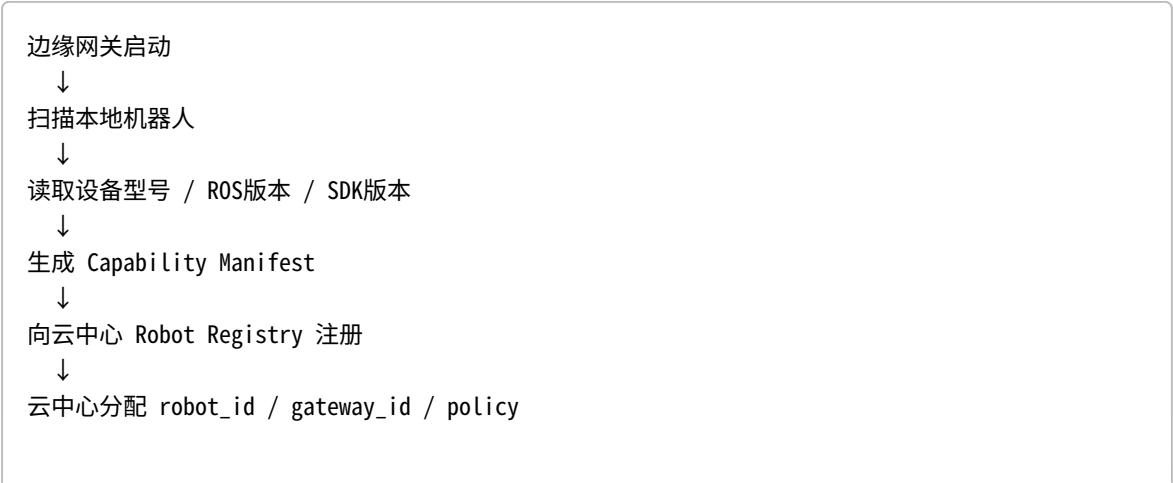
通信内容	推荐协议
任务下发	gRPC / HTTP
实时状态	WebSocket / Zenoh
遥测数据	Zenoh / MQTT / Kafka Gateway
Agent 工具调用	MCP
视频流	WebRTC / RTSP / GB28181 可选
文件上传	HTTP / S3 API
日志上报	HTTP Batch / Kafka

8.2 边缘网关到机器人

机器人类型	推荐协议
ROS2	DDS / rosbridge / Zenoh
ROS1	roscore / rosbridge / ros1_bridge
Unitree	SDK / UDP
自研设备	TCP / UDP / CAN / Serial
仿真	ROS2 / Gazebo Transport / Isaac Sim API

9. 核心业务流程

9.1 机器人注册流程



↓
边缘开始周期性心跳

9.2 用户下发自然语言任务

用户输入：
“让机器狗去A区巡检，发现异常拍照并返回”

↓
Agent Runtime 理解意图
↓
Mission Planner 生成任务 DAG
↓
Fleet Orchestrator 选择机器人
↓
Policy Center 做任务级校验
↓
下发到 Edge Gateway
↓
Safety Gateway 做动作级校验
↓
Local Skill Executor 执行
↓
机器人行动
↓
Trace / Memory 回传云中心

9.3 急停流程

用户点击急停 / 本地急停按钮 / 安全策略触发
↓

```
云中心发送 EMERGENCY_STOP
↓
边缘网关本地优先处理
↓
Safety Gateway 中断所有任务
↓
Local Skill Executor 停止执行
↓
Adapter 发送底层 stop / damping / zero velocity
↓
机器人停止
↓
上传急停事件
```

10. 数据库设计

10.1 核心表

```
sys_user: 用户表
sys_role: 角色表
sys_permission: 权限表

robot: 机器人表
robot_model: 机器人型号表
robot_capability: 机器人能力表
edge_gateway: 边缘网关表
robot_sensor: 传感器表

skill: 技能表
skill_version: 技能版本表
skill_capability_mapping: 技能能力映射表

mission: 任务表
mission_step: 任务步骤表
mission_event: 任务事件表

policy: 策略表
policy_rule: 策略规则表

mcp_tool: MCP工具表
mcp_tool_call: 工具调用记录表

trace: 执行轨迹表
trace_event: 执行事件表

memory: 记忆表
failure_case: 失败案例表
```


telemetry: 遥测数据表
alarm: 告警表
media_asset: 图片/视频资源表
map_asset: 地图资源表

10.2 Robot 表

```
CREATE TABLE robot (  
  id BIGSERIAL PRIMARY KEY,  
  robot_id VARCHAR(64) UNIQUE NOT NULL,  
  name VARCHAR(128) NOT NULL,  
  vendor VARCHAR(64),  
  model VARCHAR(64),  
  robot_type VARCHAR(64),  
  ros_version VARCHAR(32),  
  control_protocol VARCHAR(64),  
  edge_gateway_id VARCHAR(64),  
  online_status VARCHAR(32),  
  battery_level NUMERIC(5,2),  
  location JSONB,  
  safety_profile VARCHAR(64),  
  created_at TIMESTAMP DEFAULT now(),  
  updated_at TIMESTAMP DEFAULT now()  
);
```

10.3 Mission 表

```
CREATE TABLE mission (  
  id BIGSERIAL PRIMARY KEY,  
  mission_id VARCHAR(64) UNIQUE NOT NULL,  
  name VARCHAR(128),  
  user_id BIGINT,  
  status VARCHAR(32),  
  priority VARCHAR(32),  
  source_type VARCHAR(32),  
  original_command TEXT,  
  plan_json JSONB,  
  started_at TIMESTAMP,  
  finished_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT now()  
);
```

10.4 Skill 表

```
CREATE TABLE skill (  
  id BIGSERIAL PRIMARY KEY,
```

```
skill_id VARCHAR(64) UNIQUE NOT NULL,  
name VARCHAR(128),  
category VARCHAR(64),  
version VARCHAR(32),  
input_schema JSONB,  
output_schema JSONB,  
safety_constraints JSONB,  
supported_robot_types JSONB,  
executor_type VARCHAR(32),  
created_at TIMESTAMP DEFAULT now()  
);
```

11. 接口设计

11.1 机器人注册接口

```
POST /api/robots/register
```

请求：

```
{  
  "robot_id": "dog-001",  
  "name": "A区机器狗1号",  
  "vendor": "Unitree",  
  "model": "Go2",  
  "robot_type": "quadruped",  
  "ros_version": "ros2",  
  "control_protocol": "unitree_sdk",  
  "capabilities": ["stand", "walk", "navigate", "capture_image"],  
  "sensors": ["camera", "imu", "lidar"],  
  "edge_gateway_id": "edge-001"  
}
```

响应：

```
{  
  "success": true,  
  "robot_id": "dog-001",  
  "policy_profile": "normal_inspection_policy",  
  "heartbeat_interval": 5  
}
```

11.2 创建任务接口

`POST /api/missions`

请求:

```
{
  "command": "让机器狗去A区巡检，发现异常拍照并返回",
  "mode": "natural_language",
  "approval_required": true
}
```

响应:

```
{
  "mission_id": "mission-20260703-001",
  "status": "WAITING_APPROVAL",
  "plan": {
    "steps": [
      "stand_up",
      "navigate_to",
      "inspect_area",
      "return_home"
    ]
  }
}
```

11.3 边缘任务下发接口

`POST /edge/api/skills/execute`

请求:

```
{
  "mission_id": "mission-20260703-001",
  "step_id": "step-2",
  "robot_id": "dog-001",
  "skill": "navigate_to",
  "params": {
    "target": "area-a-entry"
  },
  "policy": {
    "max_speed": 0.6,
    "require_obstacle_check": true
  }
}
```

```
}  
}
```

响应:

```
{  
  "accepted": true,  
  "execution_id": "exec-001",  
  "status": "VALIDATING"  
}
```

12. 安全设计

12.1 分层安全

用户层: 身份认证、角色权限、操作审计
任务层: 任务审批、任务模板、区域限制
工具层: MCP 工具白名单、工具参数校验
边缘层: Safety Gateway、本地策略、本地急停
机器人层: 底层限速、急停、姿态保护、故障保护

12.2 高危动作清单

高速移动
远距离移动
进入危险区域
楼梯/坡道移动
靠近人员
关节/力控动作
越权区域巡检
夜间无人值守任务
多机器人协同靠近

高危动作必须:

强制审批
强制限速
强制记录
可随时急停
支持人工接管

13. 部署设计

13.1 云中心部署

Kubernetes / Docker Compose

|—

web-console

|—

api-gateway

|—

auth-service

|—

robot-registry-service

|—

mission-planner-service

|—

fleet-orchestrator-service

|—

skill-registry-service

|—

mcp-tool-registry-service

|—

policy-service

|—

memory-service

|—

trace-service

|—

media-service

|—

postgres

|—

redis

|—

minio

|—

kafka/nats

|—

prometheus/grafana

13.2 边缘网关部署

Edge Host / Jetson / 工控机

|—

edge-agent

|—

rosclaw-edge-runtime

|—

safety-gateway

|—

local-skill-executor

|—

ros2-adapter

|—

ros1-adapter

|—

unitree-adapter

|—

sensor-adapter

|—

telemetry-agent

|—

local-cache

|—

watchdog

13.3 推荐硬件

场景	推荐配置
云中心测试	8C16G + 1 张消费级 GPU 可选
云中心生产	16C64G 起，GPU 按模型需求配置
边缘网关轻量	Intel NUC / 工控机 / Jetson Orin Nano

场景	推荐配置
边缘网关视觉	Jetson Orin NX / AGX Orin
仿真开发	GPU 工作站

14. MVP 落地路径

阶段一：单机器人闭环

目标：打通自然语言到单台机器人执行。

范围：

1 台 ROS2 机器人或 Unitree 机器狗
云端任务创建
边缘网关接收任务
stand_up / stop / move_forward / capture_image
执行日志回传
急停可用

阶段二：多型号接入

目标：支持 ROS2 + ROS1 + Unitree。

范围：

ROS2 Adapter
ROS1 Gateway
Unitree SDK Adapter
Capability Manifest
Robot Registry
Skill Registry

阶段三：多机器人调度

目标：支持一脑多控。

范围：

Fleet Orchestrator
机器人选择
任务分配
多机器人状态监控

任务冲突检测
任务失败重试

阶段四：安全与审计强化

目标：生产可控。

范围：

Policy Center
Safety Gateway
Trace Center
人工审批
急停
操作审计
任务回放

阶段五：记忆与技能进化

目标：形成平台闭环。

范围：

Memory Center
失败案例库
技能成功率统计
技能参数优化
场景经验沉淀
仿真验证

15. 推荐最终架构选型

最终建议采用：

云中心：
自研一脑多控平台
+ OpenClaw / NemoClaw / Hermes / QwenPaw 作为 Agent Runtime
+ MCP Tool Registry
+ Mission Planner
+ Fleet Orchestrator
+ Robot Registry
+ Skill Registry
+ Policy / Trace / Memory

边缘侧：

ROSClaw Edge Runtime
+ Safety Gateway
+ Local Skill Executor
+ ROS2 Adapter
+ ROS1 Adapter
+ Unitree SDK Adapter
+ Sensor Adapter

通信层：

MCP 用于 Agent 工具调用
gRPC/HTTP 用于任务下发
WebSocket 用于实时监控
Zenoh 用于弱网和机器人分布式通信
RTSP/WebRTC 用于视频流

机器人侧：

ROS2 主线
ROS1 兼容
Unitree SDK 适配
自研协议封装
仿真机器人验证

16. 关键结论

1. ROSClaw 更适合作为边缘侧 Physical AI Runtime，而不是单独作为完整云平台。
2. AgenticROS 更适合做 ROS2 快速接入插件，可以作为 ROS2 Adapter 的参考实现。
3. 一脑多控平台必须自研云中心控制平面，包括设备、任务、技能、权限、安全、调度、日志、记忆。
4. 真实机器狗控制必须云边协同，不能只靠云端大模型实时控制。
5. ROS1 设备建议通过兼容网关逐步接入，不建议作为长期主栈。
6. 所有机器人能力都应该抽象为 Skill / Capability，而不是直接暴露 ROS topic、SDK 指令或 UDP 控制码。

最终架构可以概括为：

云端大脑负责规划
边缘网关负责安全执行
机器人本体负责实时控制
Trace/Memory 负责持续学习
Skill Registry 负责能力复用
Fleet Orchestrator 负责多机协同